

Лабораторна робота №11

Тема: Перевантаження операторів

Мета: Навчитися перевантажувати оператори для реалізації зручної роботи з користувацькими типами даних у C#.

Хід роботи:

Завдання 1: Злити гілку feature/add- abstraction в основну гілку (master або main). Створити нові гілку

git checkout -b feature/add-override

Завдання 2: У базовому класі перевантажити оператори (оператори для перевантаження та дії, що повинні виконуватись, описані в таблиці).

2.1 Арифметичні оператори: наприклад, +, -, які виконуватимуть дії з числовими властивостями (наприклад, кількість, рейтинг, ціну тощо).

2.2 Оператори порівняння: наприклад, ==, <, >, щоб перевіряти рівність або порівнювати об'єкти за ключовими властивостями (наприклад, дата, рейтинг).

2.3 Комбінуючі оператори: наприклад, +=, щоб об'єднувати властивості об'єктів.

7	ShowTime, SeatNumber	+, ==, <	+: Комбінування резервів (наприклад, для групових бронювань) ==: Той самий час і місце <: Час

Завдання 3: Інтеграція з інтерфейсом користувача

3.1 Додати елементи керування (кнопки, текстові поля) для виконання операцій над об'єктами через перевантажені оператори.

3.2 Реалізувати додавання/об'єднання об'єктів через +.

3.3 Реалізувати порівняння двох об'єктів через ==, < або >.

3.4 Виводити результат операцій у DataGridView або текстове поле.

Завдання 4: Створити коміт та завантажити зміни в віддалений репозиторій.

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.F7.9.000 –Лр.11						
Змн.	Арк.	№ докум.	Підпис	Дата							
Розроб.		Янєв М.О.			Звіт з лабораторної роботи №11			Літ.	Арк.	Акрушів	
Перевір.		Українець М.О.								1	13
Реценз.								ФІКТ, гр. КІ-25-1			
Н. Контр.											
Зав.каф.		Єфіменко А.А.									

Лістинг програми:

MainWindow.xaml:

```
<Window x:Class="OOPWPFProject.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:OOPWPFProject"
        mc:Ignorable="d"
        Title="Система бронювання квитків" Height="700" Width="1200" FontSize="14"
        FontFamily="Verdana">

    <!-- Форма -->
    <Grid>

        <!-- Список бронювань -->
        <Border Grid.Column="1" Background="White" Padding="10">
            <DockPanel>

                <TextBlock DockPanel.Dock="Top" Text="Список бронювань" FontSize="24"
                FontWeight="Bold" Margin="0,0,0,20"/>
                <!-- Сортювання -->

                <Grid DockPanel.Dock="Top" Margin="0,0,0,10">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*" />
                        <ColumnDefinition Width="Auto" />
                    </Grid.ColumnDefinitions>

                    <StackPanel Orientation="Horizontal" Grid.Column="0">
                        <Label Content="Сортювати за: " VerticalAlignment="Center"/>
                        <ComboBox x:Name="SortComboBox" Width="158" Margin="0,0,10,0"
                        SelectedIndex="0">
                            <ComboBoxItem Content="Назва фільму"/>
                            <ComboBoxItem Content="Час сеансу"/>
                            <ComboBoxItem Content="Номер місця"/>
                        </ComboBox>
                        <Button Content="Сортювати" Click="SortRecords_Click"
                        Style="{StaticResource PrimaryButtonStyle}" Width="100" Height="25"/>
                    </StackPanel>

                    <StackPanel Orientation="Horizontal" Grid.Column="1"
                    HorizontalAlignment="Right">
                        <Label Content="Знайти за назвою:" VerticalAlignment="Center"
                        Margin="0,0,5,0"/>
                        <TextBox x:Name="SearchNameInput" Width="120" Height="30"
                        Margin="0,0,10,0"/>
                        <Button Content="Пошук" Click="SearchName_Click"
                        Style="{StaticResource PrimaryButtonStyle}" Width="100" Height="25"/>
                    </StackPanel>
                </Grid>

                <!-- Пошук за індексом -->
                <Grid DockPanel.Dock="Bottom" Margin="0,10,0,0">

                    <Button x:Name="OpenReservationButton" Content="Додати запис"
                    Click="OpenReservationWindow_Click" Style="{StaticResource PrimaryButtonStyle}"
                    Width="150" Height="30" HorizontalAlignment="Left" />
                    <Button x:Name="DeleteButton" Content="Видалити"
                    Click="DeleteRecord_Click" Style="{StaticResource SecondaryButtonStyle}" Width="150"
                    Height="30" Margin="165,0,0,0" HorizontalAlignment="Left" />
                    <Button x:Name="CancelButton" Content="Скасувати бронювання"
                    Click="CancelRecord_Click" Style="{StaticResource SecondaryButtonStyle}" Width="220"
                    Height="30" HorizontalAlignment="Right"/>
                </Grid>
            </DockPanel>
        </Border>
    </Grid>
```

					ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.F7.9.000 –Лр.11	Арк.
			</Grid>			2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

        <!-- DataGrid -->
        <DataGrid x:Name="BookingsDataGrid" AutoGenerateColumns="False"
IsReadOnly="True">
            <DataGrid.RowStyle>
                <Style TargetType="DataGridRow">
                    <Style.Triggers>
                        <DataTrigger Binding="{Binding IsCanceled}" Value="True">
                            <Setter Property="Foreground" Value="Gray" />
                            <Setter Property="FontStyle" Value="Italic" />
                        </DataTrigger>
                    </Style.Triggers>
                </Style>
            </DataGrid.RowStyle>

            <DataGrid.Columns>
                <DataGridTextColumn Header="Назва фільму" Binding="{Binding
MovieTitle}" Width="*" />
                <DataGridTextColumn Header="Час сеансу" Binding="{Binding
ShowTime, StringFormat=\{0:hh\\:mm\\}" Width="0.5*" />
                <DataGridTextColumn Header="Місце" Binding="{Binding
SeatDisplay}" Width="0.5*" />
                <DataGridTextColumn Header="Деталі" Binding="{Binding Details}"
Width="1.5*" />
            </DataGrid.Columns>

            <DataGrid.ContextMenu>
                <ContextMenu>
                    <MenuItem Header="Об'єднати" Click="GroupRecords_Click">
                        <MenuItem.Icon>
                            <TextBlock Width="15" Text="+" FontWeight="Bold"
TextAlignment="Center" Foreground="#007ACC" />
                        </MenuItem.Icon>
                    </MenuItem>

                    <MenuItem Header="36іг часу і місця"
Click="CompareEquality_Click">
                        <MenuItem.Icon>
                            <TextBlock Width="30" Text="==" FontWeight="Bold"
TextAlignment="Left" Foreground="#007ACC" />
                        </MenuItem.Icon>
                    </MenuItem>

                    <MenuItem Header="Який сеанс раніше"
Click="CompareTime_Click">
                        <MenuItem.Icon>
                            <TextBlock Width="30" Text="<>"
FontWeight="Bold" TextAlignment="Left" Foreground="#007ACC" />
                        </MenuItem.Icon>
                    </MenuItem>
                </ContextMenu>
            </DataGrid.ContextMenu>
        </DataGrid>
    </DockPanel>

    </Border>
</Grid>
</Window>

```

```

using System.Collections.ObjectModel;
using System.Text;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
using System.Windows.Documents;
using System.Windows.Input;
using System.Windows.Media;
using System.Windows.Media.Imaging;
using System.Windows.Navigation;
using System.Windows.Shapes;

namespace OOPWPFProject
{
    /// <summary>
    /// Interaction logic for MainWindow.xaml
    /// </summary>
    public partial class MainWindow : Window
    {
        EntityManager<Reservation> bookings = new EntityManager<Reservation>();

        public MainWindow()
        {
            InitializeComponent();
            bookings = new EntityManager<Reservation>();
            BookingsDataGrid.ItemsSource = bookings.Items;
        }

        private void OpenReservationWindow_Click(object sender, RoutedEventArgs e)
        {
            AddReservationWindow addWindow = new AddReservationWindow(bookings);
            addWindow.ShowDialog();
        }

        private void DeleteRecord_Click(object sender, RoutedEventArgs e)
        {
            var selected = BookingsDataGrid.SelectedItem as Reservation;
            if (selected != null)
            {
                MessageBoxResult result = MessageBox.Show("Видалити ці записи?",
                    "Видалення", MessageBoxButton.OKCancel, MessageBoxImage.Question);

                if (result == MessageBoxResult.OK) bookings.Remove(selected);
            }
            else
            {
                MessageBox.Show("Будь ласка, оберіть запис для видалення.", "Помилка",
                    MessageBoxButton.OK, MessageBoxImage.Warning);
            }
        }

        private void CancelRecord_Click(object sender, RoutedEventArgs e)
        {
            var selected = BookingsDataGrid.SelectedItem as Reservation;
            if (selected != null)
            {
                if (selected.IsCanceled)
                {
                    MessageBox.Show("Це бронювання вже скасовано!", "Інформація",
                        MessageBoxButton.OK, MessageBoxImage.Information);
                    return;
                }

                MessageBoxResult result = MessageBox.Show("Скасувати ці бронювання?",
                    "Скасування", MessageBoxButton.OKCancel, MessageBoxImage.Question);
            }
        }
    }
}

```

```

        selected.Cancel();
        BookingsDataGrid.Items.Refresh();
        MessageBox.Show("Бронювання успішно скасовано!", "Інформація",
        MessageBoxButton.OK, MessageBoxImage.Information);
    }
}
else
{
    MessageBox.Show("Будь ласка, оберіть запис для скасування.", "Помилка",
    MessageBoxButton.OK, MessageBoxImage.Warning);
}
}

// Метод для обробки кліку на кнопку "Сортувати"
private void SortRecords_Click(Object sender, RoutedEventArgs e)
{
    if (bookings.Items.Count == 0)
    {
        MessageBox.Show("Список записів порожній.", "Помилка",
        MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    string? sortOption = (SortComboBox.SelectedItem as
    ComboBoxItem)?.Content.ToString();

    IEnumerable<Reservation> sortedRecords = null;

    switch (sortOption)
    {
        case "Назва фільму":
            sortedRecords = bookings.Items.OrderBy(b => b.MovieTitle).ToList();
            break;

        case "Час сеансу":
            sortedRecords = bookings.Items.OrderBy(b => b.ShowTime).ToList();
            break;
        case "Номер місця":
            sortedRecords = bookings.Items.OrderBy(b => b.SeatNumbers).ToList();
            break;
    }

    if (sortedRecords != null)
    {
        bookings.Clear();
        foreach (var r in sortedRecords)
        {
            bookings.Add(r);
        }
    }
}

// Метод для обробки кліку на кнопку "Пошук"
private void SearchName_Click(Object sender, RoutedEventArgs e)
{
    string nameInput = SearchNameInput.Text;
    if (string.IsNullOrEmpty(nameInput)) { MessageBox.Show("Будь ласка, введіть
назву для пошуку.", "Помилка", MessageBoxButton.OK, MessageBoxImage.Warning); return; }

    EntityManager<Reservation>? foundBooking = new EntityManager<Reservation>();
    int foundCount = 0;

    foreach (var record in bookings.Items)
    {
        if (record.MovieTitle.ToLower() == nameInput.ToLower())
        {
            foundBooking.Add(record);
            foundCount++;
        }
    }
}

```

```

    }

    }
    if (foundCount > 0)
    {
        MessageBox.Show($"Знайдено записів: {foundCount}.", "Пошук",
        MessageBoxButton.OK, MessageBoxImage.Information);
        foundBooking.DisplayAll();
    }
    else
    {
        MessageBox.Show("Записів не знайдено.", "Пошук", MessageBoxButton.OK,
        MessageBoxImage.Information);
    }
}
// Метод для обробки групування бронювань
private void GroupRecords_Click(object sender, RoutedEventArgs e)
{
    if (BookingsDataGrid.SelectedItems.Count < 2)
    {
        MessageBox.Show("Будь ласка, оберіть щонайменше 2 записи для об'єднання (Затисніть Ctrl).", "Помилка", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }
    try
    {
        var selectedRecords =
        BookingsDataGrid.SelectedItems.Cast<Reservation>().ToList();
        Reservation combinedRecords = selectedRecords[0];

        for (int i = 1; i < selectedRecords.Count; i++)
        {
            combinedRecords += selectedRecords[i];
        }

        foreach (var record in selectedRecords)
        {
            bookings.Remove(record);
        }

        bookings.Add(combinedRecords);
        BookingsDataGrid.Items.Refresh();

        MessageBox.Show("Бронювання успішно об'єднано!", "Помилка",
        MessageBoxButton.OK, MessageBoxImage.Information);
    }
    catch (ArgumentException ex)
    {
        MessageBox.Show(ex.Message, "Помилка об'єднання", MessageBoxButton.OK,
        MessageBoxImage.Error);
    }
}
// Метод для порівняння бронювань на час та місце
private void CompareEquality_Click(object sender, RoutedEventArgs e)
{
    if (BookingsDataGrid.SelectedItems.Count != 2)
    {
        MessageBox.Show("Будь ласка, оберіть рівно 2 записи для порівняння.",
        "Помилка", MessageBoxButton.OK, MessageBoxImage.Warning);

        return;
    }
    var selectedRecords =
    BookingsDataGrid.SelectedItems.Cast<Reservation>().ToList();
    if (selectedRecords[0] == selectedRecords[1])
    {

```

```

        MessageBox.Show("Ці записи мають однаковий час та місце.", "Порівняння",
        MessageBoxButton.OK, MessageBoxImage.Information);
    }
    else
    {
        MessageBox.Show("Ці записи не мають однаковий час та місце.",
        "Порівняння", MessageBoxButton.OK, MessageBoxImage.Information);
    }
}

// Метод для перевірки, яке бронювання раніше
private void CompareTime_Click(object sender, RoutedEventArgs e)
{
    if (BookingsDataGrid.SelectedItems.Count != 2)
    {
        MessageBox.Show("Будь ласка, оберіть рівно 2 записи для порівняння.",
        "Увага", MessageBoxButton.OK, MessageBoxImage.Warning);
        return;
    }

    var selected = BookingsDataGrid.SelectedItems.Cast<Reservation>().ToList();

    if (selected[0] < selected[1])
    {
        MessageBox.Show($"Сеанс '{selected[0].MovieTitle}' відбувається раніше,
        ніж '{selected[1].MovieTitle}'.", "Порівняння часу (<)", MessageBoxButton.OK,
        MessageBoxImage.Information);
    }
    else if (selected[0] > selected[1])
    {
        MessageBox.Show($"Сеанс '{selected[1].MovieTitle}' відбувається раніше,
        ніж '{selected[0].MovieTitle}'.", "Порівняння часу (<)", MessageBoxButton.OK,
        MessageBoxImage.Information);
    }
    else
    {
        MessageBox.Show("Ці сеанси відбуваються одночасно.", "Порівняння часу",
        MessageBoxButton.OK, MessageBoxImage.Information);
    }
}
}
}

```

Reservation.cs:

```

using System;
using System.Collections.Generic;
using System.DirectoryServices.ActiveDirectory;
using System.IO.Packaging;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace OOPWPFProject
{
    public abstract class AbstractReservation
    {
        public string MovieTitle { get; set; }
        public TimeSpan ShowTime { get; set; }
        public List<int> SeatNumbers { get; set; }
        public string SeatDisplay => string.Join(", ", SeatNumbers);

        public string Format { get; set; }
        public bool IsCanceled { get; set; }
    }
}

```

```

{
    public Reservation(string movieTitle, TimeSpan showtime, int seatNumber, string
format, bool isCanceled = false)
    {
        if (string.IsNullOrEmpty(movieTitle)) throw new ArgumentException("Назва
фільму не може бути порожньою.");
        if (seatNumber < 1 || seatNumber > 30) throw new ArgumentException("Номер
місця не може бути порожнім.");
        MovieTitle = movieTitle;
        ShowTime = showtime;
        SeatNumbers = new List<int> { seatNumber };
        Format = format;
        IsCanceled = isCanceled;
    }
    // Конструктор для групових бронювань
    public Reservation(string movieTitle, TimeSpan showtime, List<int> seatNumber,
string format, bool isCanceled = false)
    {
        if (string.IsNullOrEmpty(movieTitle)) throw new ArgumentException("Назва
фільму не може бути порожньою.");
        if (seatNumber.Count == 0) throw new ArgumentException("Номер місця не може
бути порожнім.");
        MovieTitle = movieTitle;
        ShowTime = showtime;
        SeatNumbers = new List<int> (seatNumber);
        Format = format;
        IsCanceled = isCanceled;
    }

    // Перевантаження оператора +
    public static Reservation operator +(Reservation r1, Reservation r2)
    {
        if ((r1.MovieTitle == r2.MovieTitle) && (r1.ShowTime == r2.ShowTime) &&
(r1.Format == r2.Format)){
            var groupedSeats = new List<int> { };
            groupedSeats.AddRange(r1.SeatNumbers);
            groupedSeats.AddRange(r2.SeatNumbers);
            groupedSeats = groupedSeats.Distinct().OrderBy(x => x).ToList();

            // Перевірка на VIP
            bool isVip1 = r1 is VIPReservation;
            bool isVip2 = r2 is VIPReservation;

            if (isVip1 || isVip2)
            {
                var vip1 = r1 as VIPReservation;
                var vip2 = r2 as VIPReservation;

                bool combinedLounge = (vip1?.LoungeAccess == true) &&
(vip2?.LoungeAccess == true);
                bool combinedSnacks = (vip1?.ComplementarySnacks == true) &&
(vip2?.ComplementarySnacks == true);

                string? combinedWishes = $"{vip1?.AdditionalWishes}
{vip2?.AdditionalWishes}".Trim();

                return new VIPReservation(r1.MovieTitle, r1.ShowTime, groupedSeats,
r1.Format, combinedLounge, combinedSnacks, combinedWishes);
            }
            return new Reservation(r1.MovieTitle, r1.ShowTime, groupedSeats,
r1.Format);
        }
        else
        {
            throw new ArgumentException("Сеанси не збігаються!");
        }
    }
}

```

Арк.

8

```

    }
}

// Перевантаження операторів > та <
public static bool operator >(Reservation r1, Reservation r2)
{
    return r1.ShowTime > r2.ShowTime;
}
public static bool operator <(Reservation r1, Reservation r2)
{
    return r1.ShowTime < r2.ShowTime;
}

// Перевантаження операторів == та !=
public static bool operator ==(Reservation r1, Reservation r2)
{
    if (ReferenceEquals(r1, r2)) return true;
    if (r1 is null || r2 is null) return false;

    return r1.ShowTime == r2.ShowTime &&
r1.SeatNumbers.Intersect(r2.SeatNumbers).Any();
}

public static bool operator !=(Reservation r1, Reservation r2)
{
    return !(r1 == r2);
}

public override bool Equals(object obj)
{
    if (obj is Reservation other)
        return this == other;
    return false;
}

public override int GetHashCode()
{
    return ShowTime.GetHashCode();
}

public virtual string GetReservationDetails()
{
    return IsCanceled ? $"[СКАСОВАНО] Формат: {Format}" : $"Формат: {Format}";
}
public string Details => GetReservationDetails();

public void Cancel()
{
    IsCanceled = true;
}
}

```

```

public class VIPReservation : Reservation
{

```

```

    public bool LoungeAccess { get; set; }
    public bool ComplementarySnacks { get; set; }
    public string AdditionalWishes { get; set; }

```

```

    public VIPReservation(string title, TimeSpan time, int seat, string format, bool
lounge, bool snacks, string wishes, bool isCanceled = false): base(title, time, seat,
format, isCanceled)
{

```

ЖИТОМИРСЬКА ПОЛІТЕХНІКА.26.F7.9.000 –Лр.11

Арк.

9

```

        LoungeAccess = lounge;
        ComplementarySnacks = snacks;
        AdditionalWishes = wishes;
    }
    // Конструктор для групових VIP-бронювань
    public VIPReservation(string title, TimeSpan time, List<int> seats, string
format, bool lounge, bool snacks, string wishes, bool isCanceled = false) : base(title,
time, seats, format, isCanceled)
    {
        LoungeAccess = lounge;
        ComplementarySnacks = snacks;
        AdditionalWishes = wishes;
    }
    public override string GetReservationDetails()
    {
        string lounge = LoungeAccess ? "Так" : "Ні";
        string snacks = ComplementarySnacks ? "Так" : "Ні";
        string wishes = string.IsNullOrEmpty(AdditionalWishes) ? "-" :
AdditionalWishes ;
        return IsCanceled ? $"[СКАСОВАНО] Формат: {Format}, VIP-зона: {lounge},
Закуси: {snacks}, Додатково:{wishes}" : $"Формат: {Format}, VIP-зона: {lounge}, Закуси:
{snacks}, Додатково:{wishes}";
    }
}
}

```

Система бронювання квитків

— □ ×

Список бронювань

Сортувати за:

Назва фільму ▾

Сортувати

Знайти за назвою:

Пошук

Назва фільму	Час сеансу	Місце	Деталі
+ Об'єднати = Збіг часу і місця <> Який сеанс раніше			

Додати запис

Видалити

Скасувати бронювання

Рис. 1 – Форма та контекстне меню.

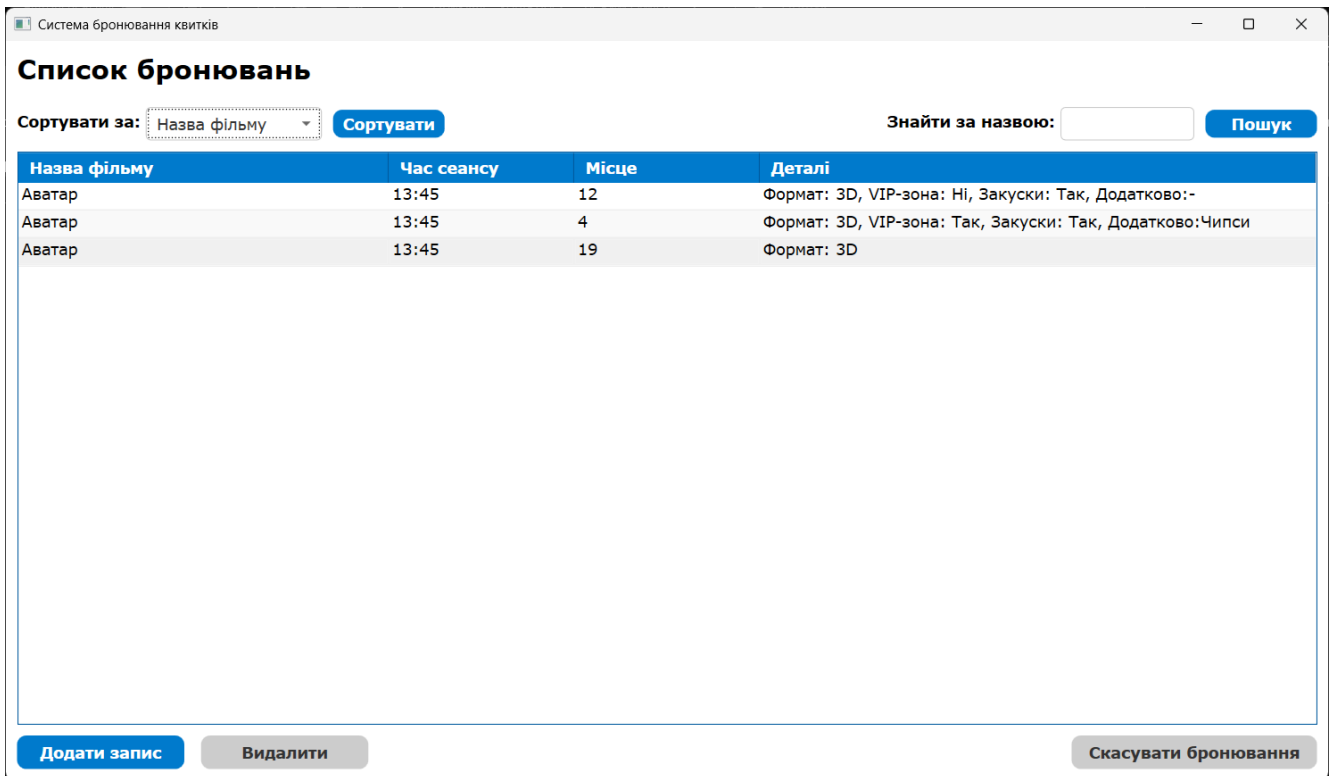


Рис. 2 – Першочерговий список.

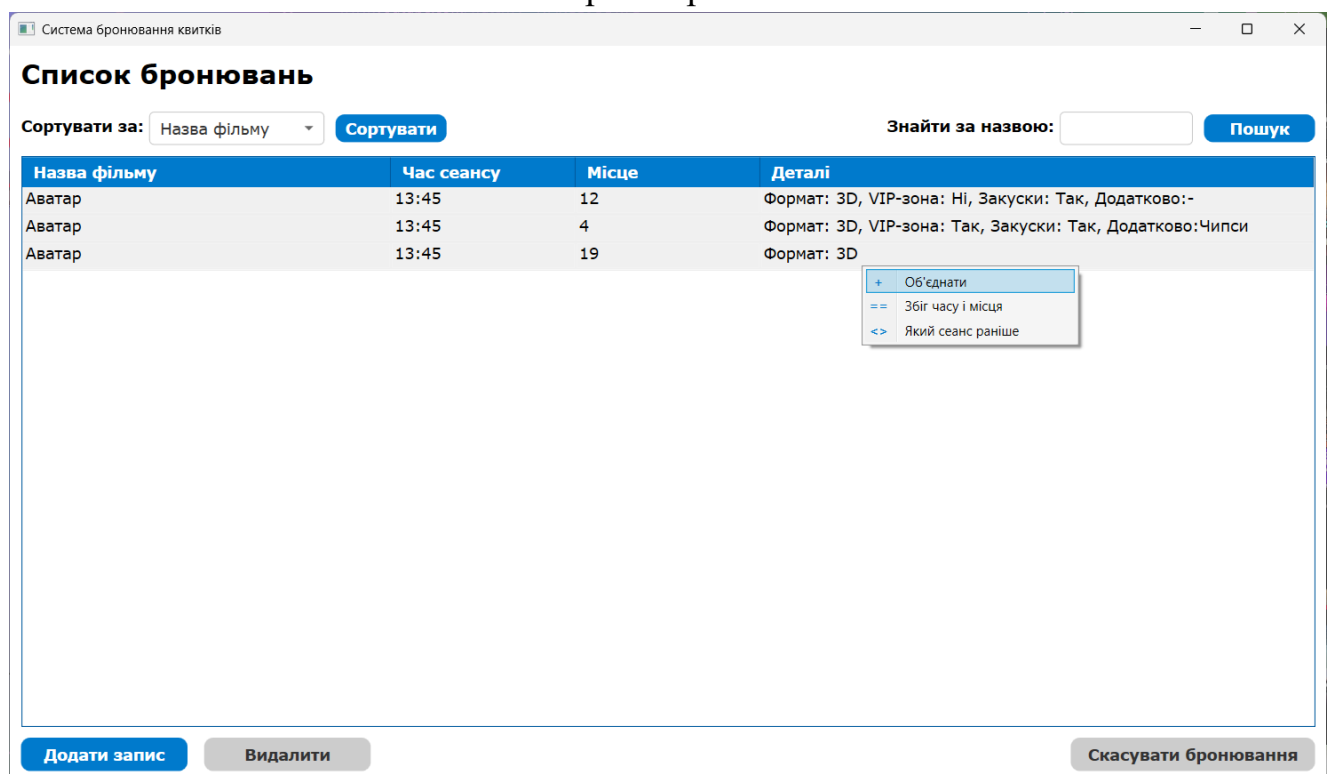


Рис. 3 – Виділення та обирання пункту «Об'єднати» в контекстному меню.

Система бронювання квитків

Список бронювань

Сортувати за: Назва фільму

Сортувати

Знайти за назвою:

Пошук

Назва фільму	Час сеансу	Місце	Деталі
Аватар	13:45	4, 12, 19	Формат: 3D, VIP-зона: Ні, Закуски: Ні, Додатково:Чипси

Додати запис

Видалити

Скасувати бронювання

Рис. 4 – Результат об’єднання 3 окремих бронювань в одне.

Система бронювання квитків

Список бронювань

Сортувати за: Назва фільму

Сортувати

Знайти за назвою:

Пошук

Назва фільму	Час сеансу	Місце	Деталі
Аватар	13:45	13	Формат: 3D, VIP-зона: Ні, Закуски: Так, Додатково:-
Тачки	12:55	7	Формат: 2D
Назад у майбутнє	12:55	7	Формат: 2D

Порівняння

Ці записи мають однаковий час та місце.

ОК

Рис. 5 – Результат перевірки на збіг часу та місця сеансів.

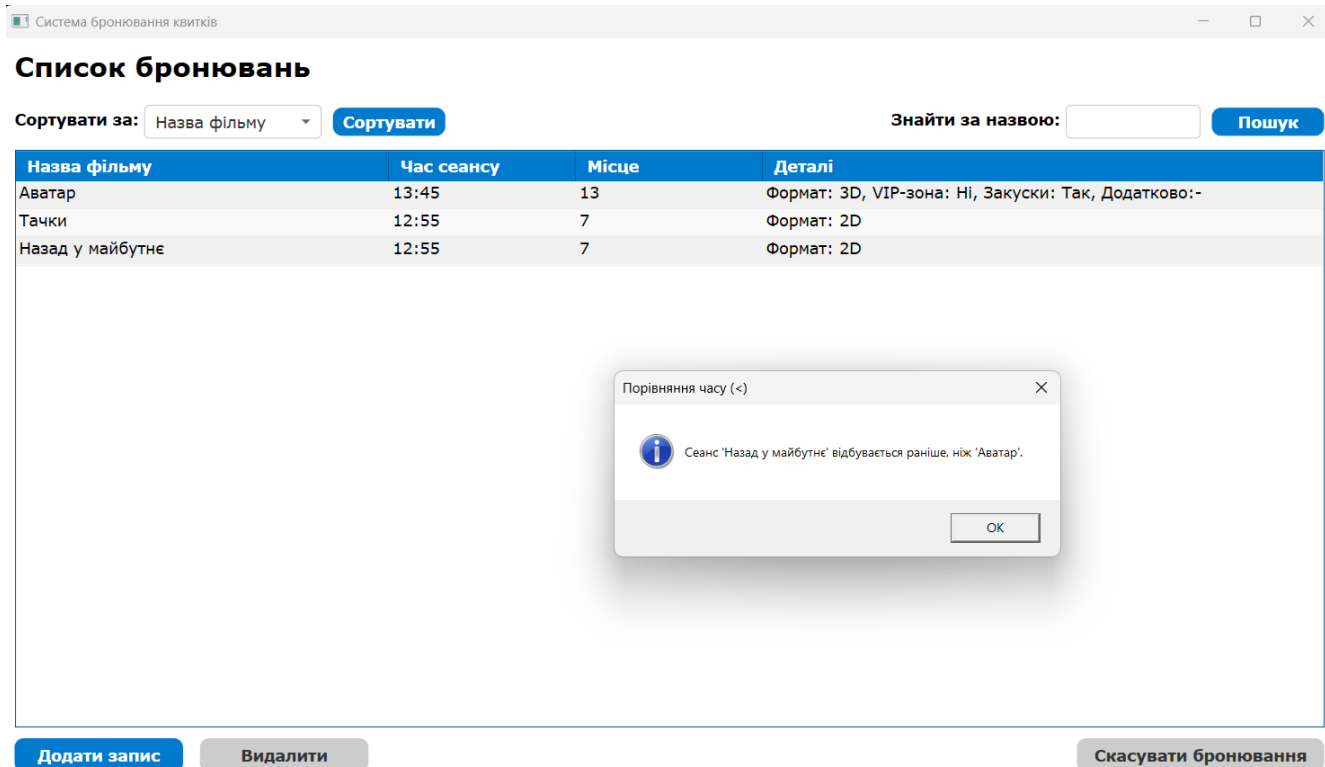


Рис. 6 – Результат порівняння, який сеанс стоїть раніше у графіку.

Висновок: У ході виконання цієї лабораторної роботи я навчився переважувати оператори для реалізації зручної роботи з користувацькими типами даних у C#. Реалізував контекстне меню з можливістю вибору окремих функцій операцій над бронюваннями.